САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №4

«Технический дизайн микросервисной архитектуры»

Выполнил:

Федоров Даниил

Группа:

K3440

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Разделить монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service
2. Спроектировать микросервисную архитектуру: отразить взаимосвязь между микросервисами, описать способы их взаимодействия друг с другом
3. Спроектировать разделение вашей БД на обособленные БД
4. Спроектировать и описать эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI
5. Описать варианты запросов и ответов, задокументировать возможные ошибки
6. Сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разеделения монолитного бэкенд-приложения на отдельные микросервисы

Ход работы

Исходное приложение

В текущей версии приложения реализованы следующие основные функциональные области:

- регистрация и авторизация пользователей с ролями соискателя и работодателя;
- управление профилем соискателя и его резюме;
- управление профилем работодателя;
- работа со справочниками опыта, отраслей и навыков;
- просмотр, создание, изменение и удаление вакансий;
- создание, просмотр и удаление откликов на вакансии;
- работа с сопроводительными письмами.

Все функциональные области реализованы внутри одного приложения и используют единую базу данных. Между сущностями используются связи TypeORM и внешние ключи PostgreSQL.

Целевая архитектура (требования)

1) БД будет делиться по принципу **Database-per-service.** Каждый микросервис имеет собственную обособленную базу данных. Другие микросервисы не будут иметь прямого доступа к её таблицам.
2) Каждый микросервис будет отыечать только за определенную функциональную чать приложения.
3) Т.К на данном этапе не будет использоваться брокер сообщений, то способом взаимодействия сервисов будет синхронное HTTP REST API
4) После разделения на микросервисы, желательно сохранить функции публичных эндпоинтов

Микросервисы

Микросервис	**Ответственность**	**Основные сущности**
User Service	Авторизация, пользователи, профили соискателей и работодателей, резюме	User, Seeker, Employer, Resume, Education, ResumeExperience, ResumeSkill
Reference Service	Централизованное хранение справочных данных	Skill, Industry, ExperienceOption
Vacancy Service	Создание, поиск, изменение и удаление вакансий	Vacancy, VacancySkill
Response Service	Работа с откликами и сопроводительными письмами	ApplicationResponse, CoverLetter

User Service

User Service отвечает за учётные записи и пользовательские данные.

В сервис переносятся операции регистрации и авторизации, работа с профилями соискателей и работодателей, а также работа с резюме.

Сервис является владельцем данных о пользователях, поэтому только он имеет прямой доступ к таблицам пользователей, профилей и резюме.

Навыки резюме сохраняются в виде идентификаторов навыков. Сами данные о навыках находятся в Reference Service.

Например, запись связи резюме с навыком содержит resume_id и skill_id. Поле resume_id может являться внешним ключом внутри базы User Service, поскольку сущность Resume принадлежит этому же сервису. Поле skill_id является обычным числовым значением и не имеет внешнего ключа на таблицу другого сервиса.

Reference Service

Reference Service отвечает за справочные данные приложения:

- навыки;
- отрасли;
- варианты требуемого опыта работы.

Справочники используются несколькими другими микросервисами. Например, навыки используются одновременно в резюме и вакансиях.

Выделение справочников в отдельный сервис позволяет избежать ситуации, при которой User Service или Vacancy Service вынуждены обращаться напрямую к чужой базе данных.

Сервис также отвечает за первоначальное заполнение справочников начальными значениями.

3.3 Vacancy Service

Vacancy Service отвечает за весь жизненный цикл вакансии:

создание вакансии, получение списка вакансий, фильтрацию, сортировку, пагинацию, получение подробной информации, изменение и удаление вакансии.

После разделения системы сущность Vacancy больше не содержит ORM-связей с Employer, Industry и ExperienceOption, поскольку эти данные принадлежат другим сервисам.

Вместо связей сохраняются идентификаторы:

employer_id, industry_id, experience_id.

Аналогично, таблица навыков вакансии содержит идентификатор вакансии и идентификатор навыка. Данные самого навыка находятся в Reference Service.

Если Vacancy Service необходимо проверить существование работодателя или получить его название компании, он обращается к User Service через REST API. Для проверки и получения справочных значений используется API Reference Service.

Response Service

Response Service отвечает за отклики соискателей на вакансии и сопроводительные письма.

Сервис хранит идентификаторы связанных объектов, находящихся в других микросервисах:

vacancy_id, resume_id, seeker_id и employer_id.

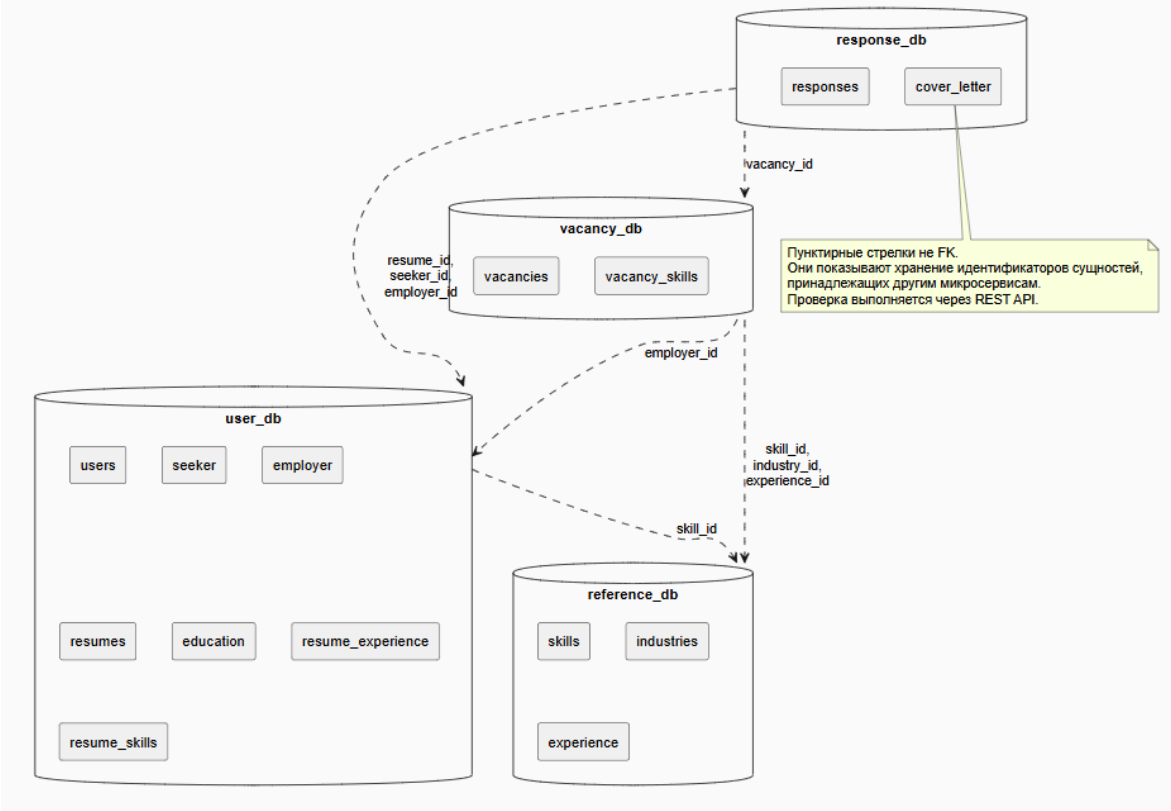
При создании отклика Response Service должен проверить существование вакансии через Vacancy Service, а также проверить существование резюме и его принадлежность текущему соискателю через User Service.

Хранение seeker_id и employer_id непосредственно в записи отклика является допустимой денормализацией. Это позволяет выполнять проверку прав доступа и фильтрацию откликов без дополнительного межсервисного запроса для каждой операции.

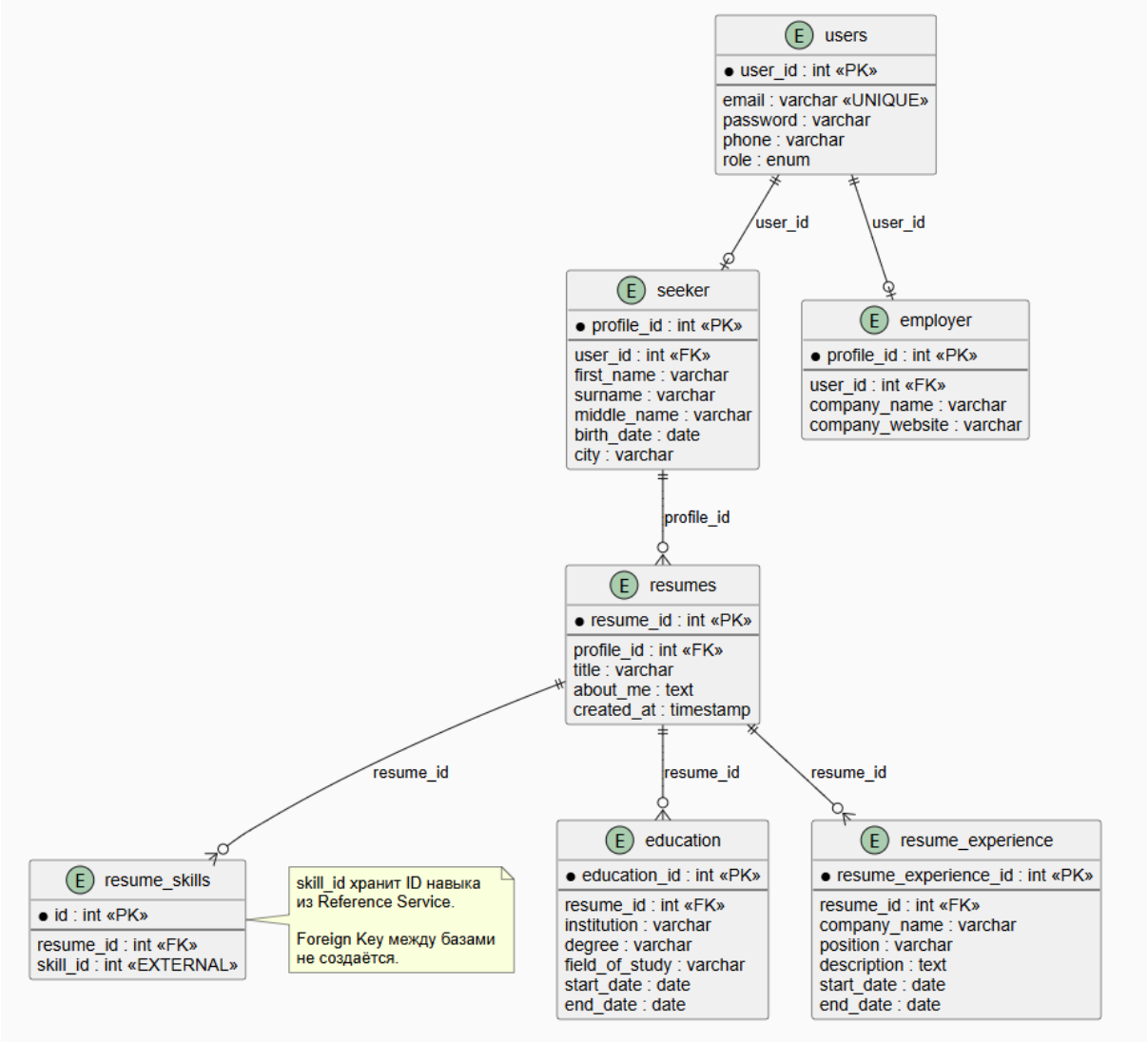
Сопроводительное письмо остаётся локальной сущностью Response Service и связано с откликом внутри одной базы данных.

Разделение БД

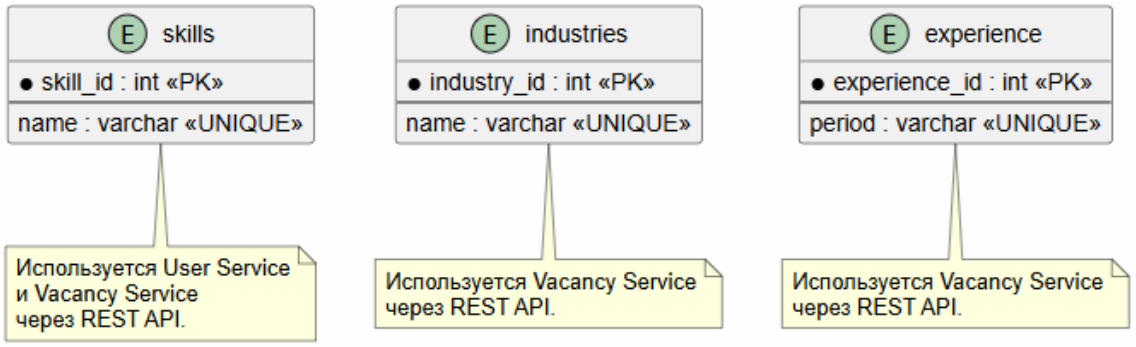
База данных	**Владелец**
user_db	User Service
reference_db	Reference Service
vacancy_db	Vacancy Service
response_db	Response Service



БД User Service



БД Reference Service

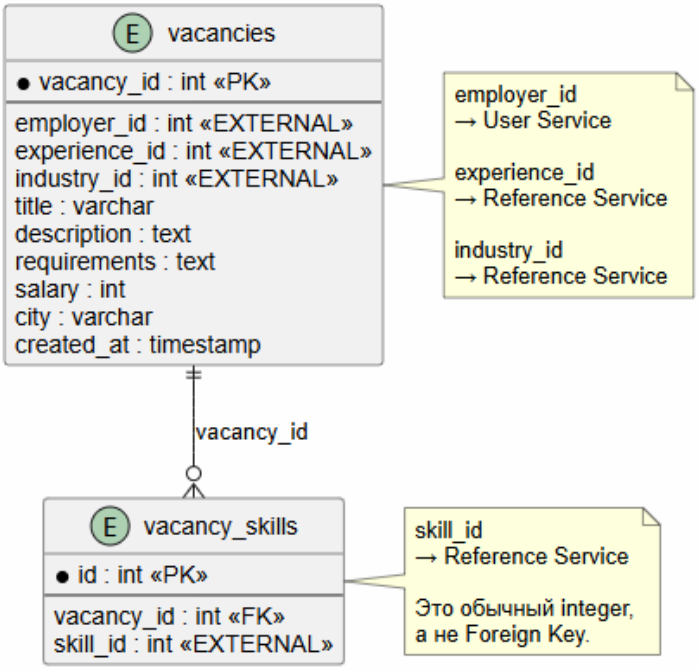


Между данными таблицами и сущностями других сервисов внешние ключи отсутствуют. Другие сервисы хранят только значения:

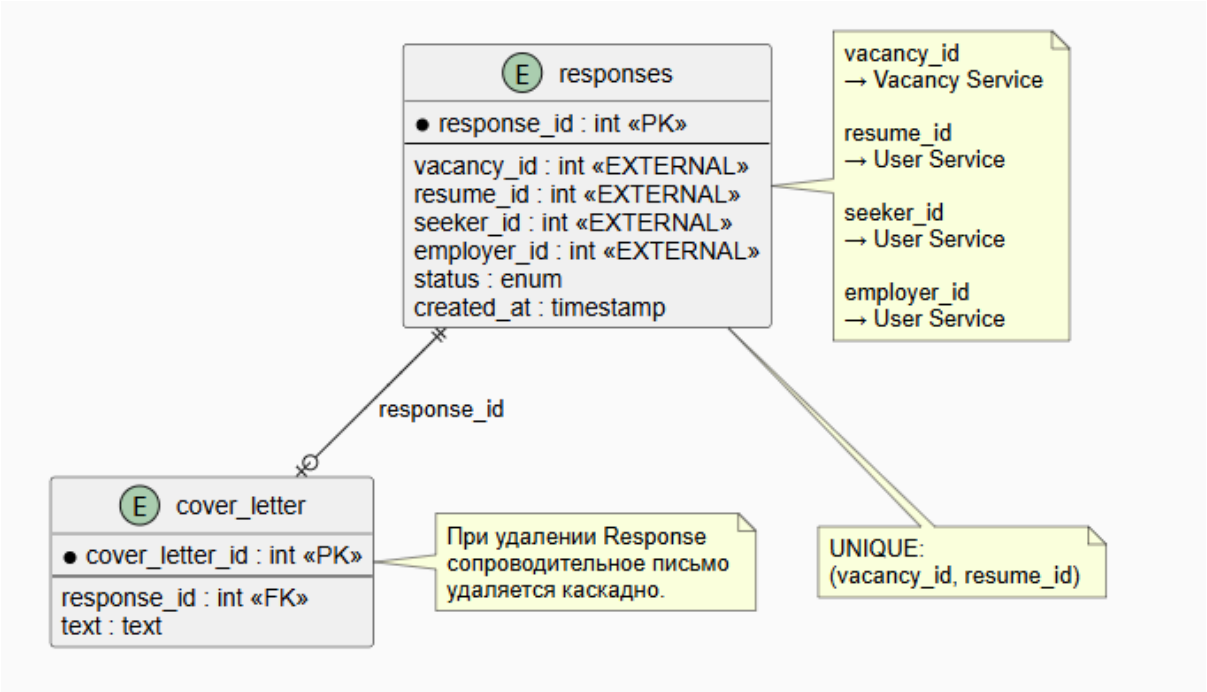
skill_id = 1

Если сервису необходимо получить название навыка или проверить его существование, он выполняет HTTP-запрос к Reference Service.

БД Vacancy Service



БД Response Service



Межсервисное взаимодействие

Так как после разделения монолитного приложения каждый микросервис получает собственную базу данных. Прямые ORM-связи между сущностями, принадлежащими разным сервисам, удаляются.

Если одному сервису необходимы данные другого сервиса, используется синхронное HTTP REST-взаимодействие.

В связи с этим можно выделить несколько основных направлений взаимодействия:

- User Service использует Reference Service для проверки навыков резюме;
- Vacancy Service использует User Service для проверки работодателей;
- Vacancy Service использует Reference Service для проверки и получения справочных данных;
- Response Service использует User Service для проверки резюме и получения данных соискателя;

- Response Service использует Vacancy Service для проверки вакансии и получения информации о ней.

Endpoints

Для обеспечения работы системы необходимо реализовать следующий минимальный набор внутренних endpoint-ов:

Сервис	**Метод**	**Endpoint**	**Назначение**
User Service	GET	/internal/employers/{employerId}	Получение и проверка работодателя
User Service	GET	/internal/resumes/{resumeId}	Получение одного резюме и его владельца
User Service	POST	/internal/resumes/resolve	Получение нескольких резюме
Reference Service	POST	/internal/reference/resolve	Проверка и получение справочных данных
Vacancy Service	GET	/internal/vacancies/{vacancyId}	Получение одной вакансии
Vacancy Service	POST	/internal/vacancies/resolve	Получение нескольких вакансий

Для публичных endpoint-ов продолжает использоваться JWT-аутентификация. JWT содержит: userId, role , profileId

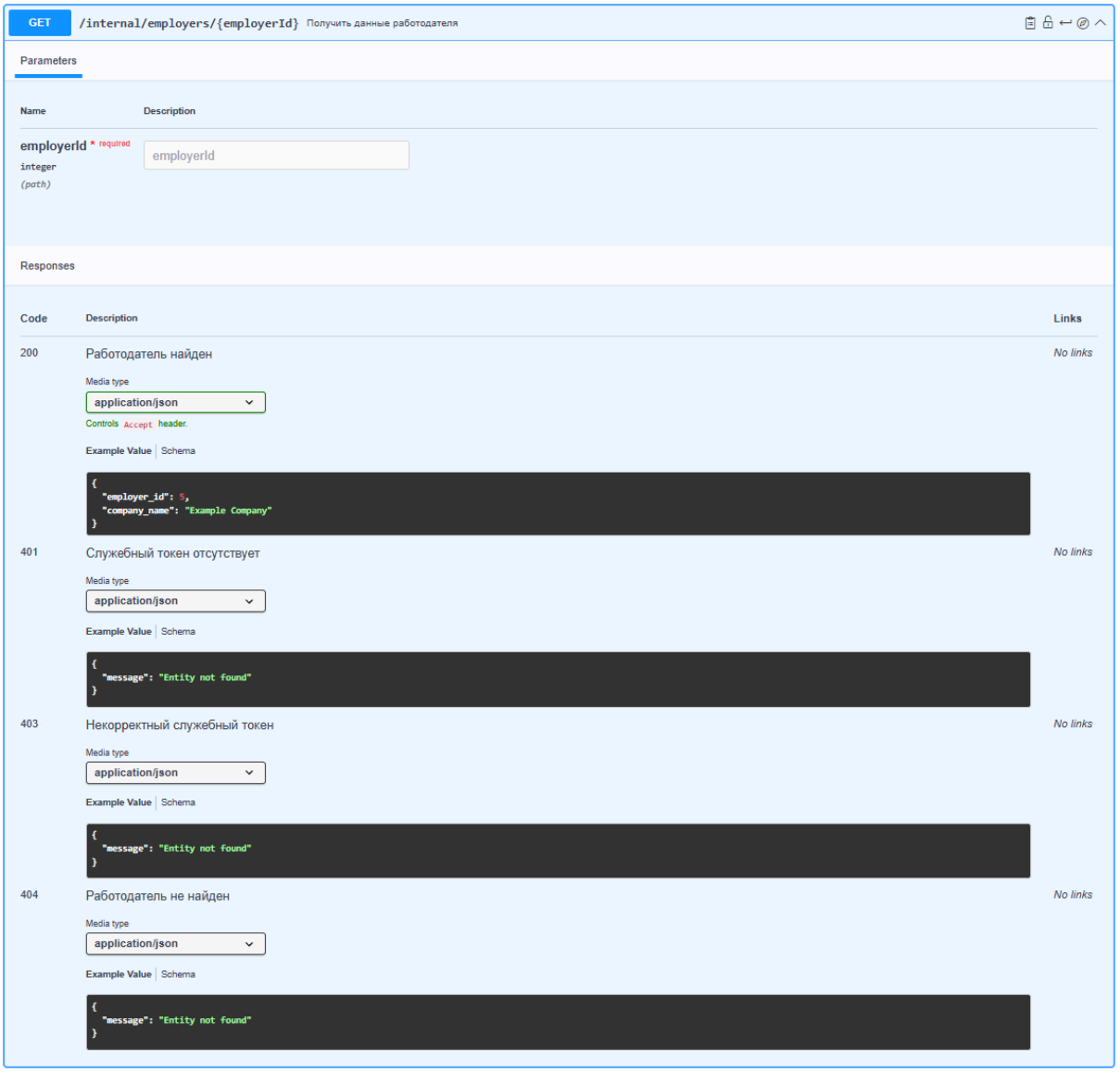
Для защиты внутренних endpoint-ов используется отдельный служебный токен. Каждый межсервисный запрос должен содержать заголовок:

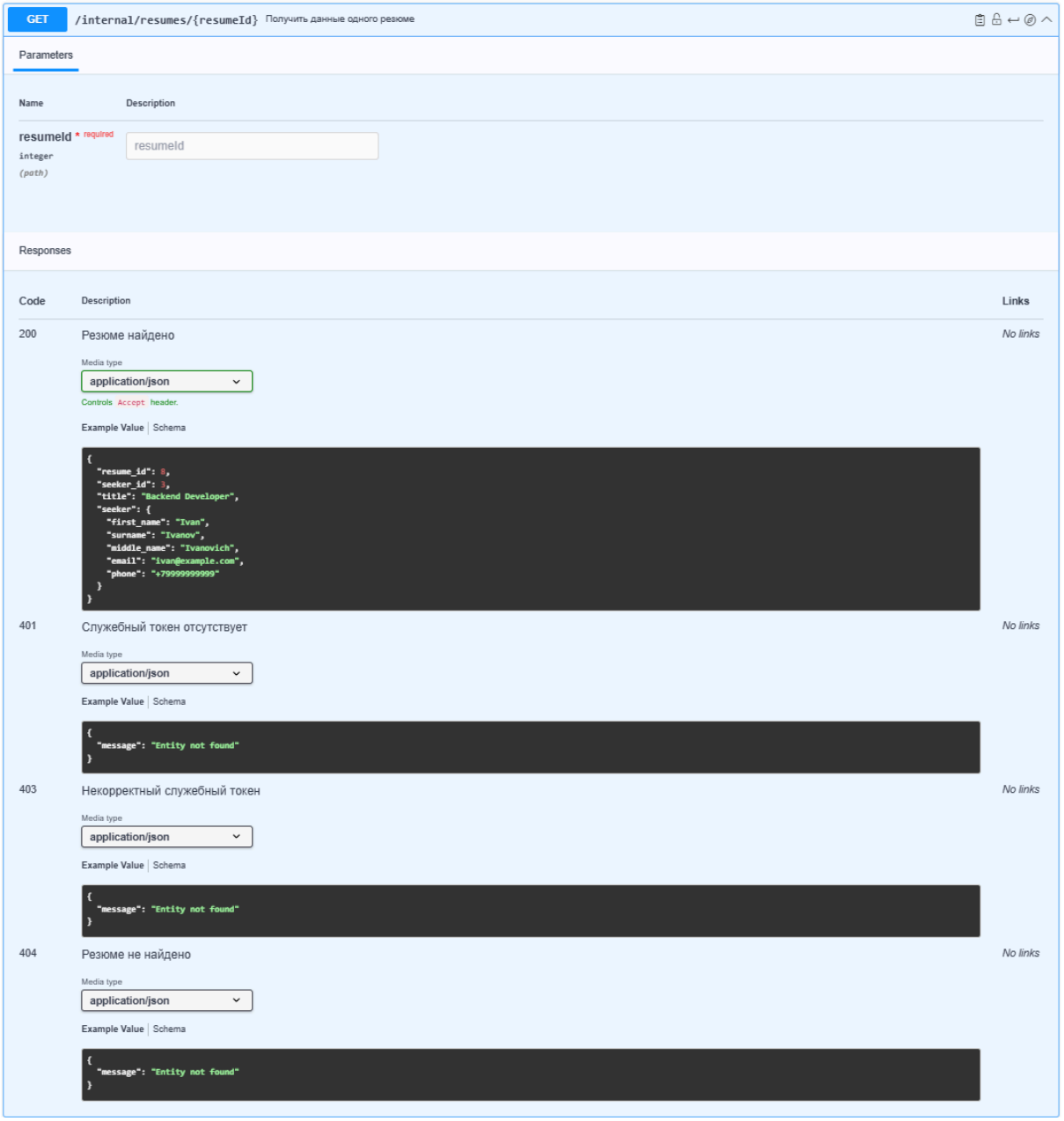
X-Service-Token: <internal-service-token>

Запросы к /internal/* без корректного служебного токена должны отклоняться.

Open API формат и ошибки

Далее будут перечислены эндпоинты в формате open api в соответствии с таблицей выше.




GET
/internal/resumes/{resumeId}
Получить данные одного резюме
Parameters
Name
Description
resumeId * required
integer
(path)
resumeId
Responses
Code
Description
Links
200
Резюме найдено
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"resume_id": 8,
"seeker_id": 3,
"title": "Backend Developer",
"seeker": {
"first_name": "Ivan",
"surname": "Ivanov",
"middle_name": "Ivanovich",
"email": "ivan@example.com",
"phone": "+79999999999"
}
}
401
Служебный токен отсутствует
No links
Media type
application/json
Example Value | Schema
{
"message": "Entity not found"
}
403
Некорректный служебный токен
No links
Media type
application/json
Example Value | Schema
{
"message": "Entity not found"
}
404
Резюме не найдено
No links
Media type
application/json
Example Value | Schema
{
"message": "Entity not found"
}

POST `/internal/resumes/resolve` Получить информацию о нескольких резюме

Parameters

No parameters

Request body required — application/json

Example Value | Schema

```
{
  "resume_ids": [
    4,
    8,
    10
  ]
}
```

Responses

Code	Description	Links
200	Данные резюме получены Media type: application/json Controls `Accept` header. Example Value | Schema	*No links*

```
{
  "items": [
    {
      "resume_id": 8,
      "seeker_id": 3,
      "title": "Backend Developer",
      "seeker_full_name": "Ivanov Ivan Ivanovich"
    }
  ]
}
```

400	Некорректный список идентификаторов Media type: application/json Example Value | Schema	*No links*

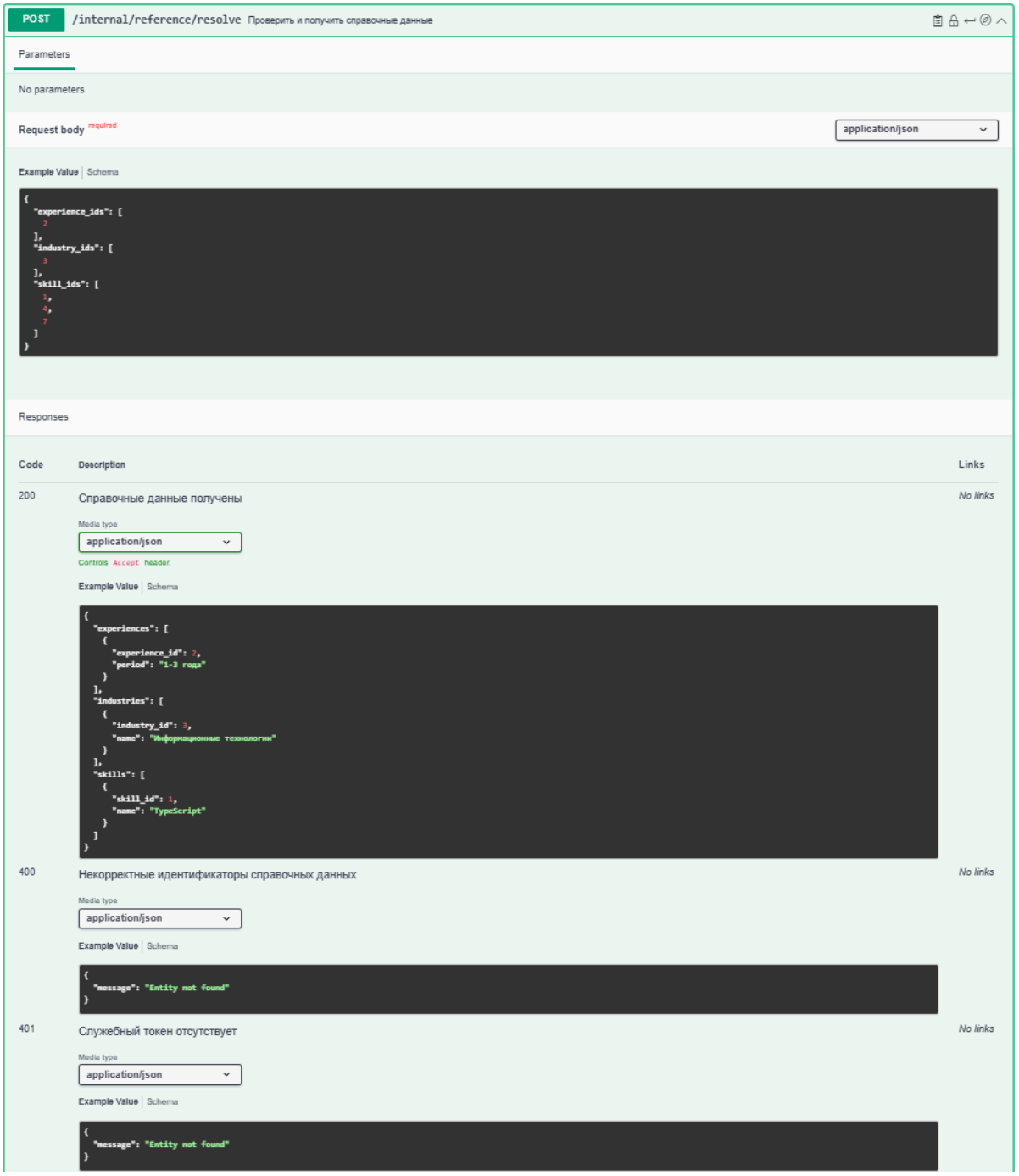
```
{
  "message": "Entity not found"
}
```

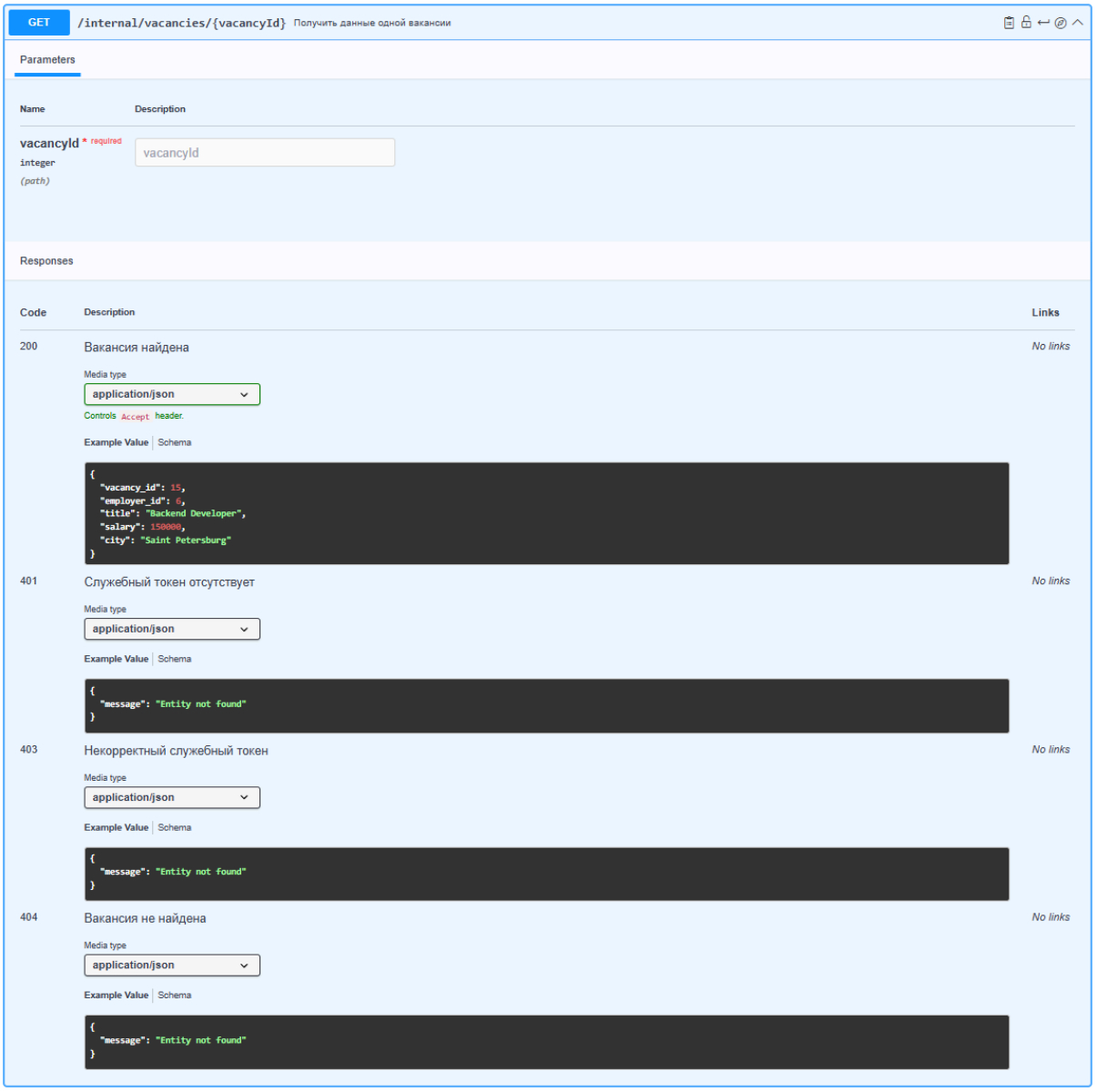
401	Служебный токен отсутствует Media type: application/json Example Value | Schema	*No links*

```
{
  "message": "Entity not found"
}
```

403	Некорректный служебный токен Media type: application/json Example Value | Schema	*No links*

```
{
  "message": "Entity not found"
}
```


POST
/internal/reference/resolve
Проверить и получить справочные данные
Parameters
No parameters
Request body
required
application/json
Example Value | Schema
{
"experience_ids": [
2
],
"industry_ids": [
3
],
"skill_ids": [
1,
4,
7
]
}
Responses
Code
Description
Links
200
Справочные данные получены
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"experiences": [
{
"experience_id": 2,
"period": "1-3 года"
}
],
"industries": [
{
"industry_id": 3,
"name": "Информационные технологии"
}
],
"skills": [
{
"skill_id": 1,
"name": "TypeScript"
}
]
}
400
Некорректные идентификаторы справочных данных
No links
Media type
application/json
Example Value | Schema
{
"message": "Entity not found"
}
401
Служебный токен отсутствует
No links
Media type
application/json
Example Value | Schema
{
"message": "Entity not found"
}


GET
/internal/vacancies/{vacancyId}
Получить данные одной вакансии
Parameters
Name
Description
vacancyId * required
integer
(path)
vacancyId
Responses
Code
Description
Links
200
Вакансия найдена
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"vacancy_id": 15,
"employer_id": 6,
"title": "Backend Developer",
"salary": 150000,
"city": "Saint Petersburg"
}
401
Служебный токен отсутствует
No links
Media type
application/json
Example Value | Schema
{
"message": "Entity not found"
}
403
Некорректный служебный токен
No links
Media type
application/json
Example Value | Schema
{
"message": "Entity not found"
}
404
Вакансия не найдена
No links
Media type
application/json
Example Value | Schema
{
"message": "Entity not found"
}

POST /internal/vacancies/resolve Получить данные нескольких вакансий

Parameters

No parameters

Request body required

application/json

Example Value | Schema

```
{
  "vacancy_ids": [
    10,
    15,
    22
  ]
}
```

Responses

Code	Description	Links
200	Данные вакансий получены Media type: application/json Controls Accept header. Example Value | Schema `{ "items": [ { "vacancy_id": 15, "employer_id": 6, "title": "Backend Developer" } ] }`	No links
400	Некорректный список идентификаторов Media type: application/json Example Value | Schema `{ "message": "Entity not found" }`	No links
401	Служебный токен отсутствует Media type: application/json Example Value | Schema `{ "message": "Entity not found" }`	No links
403	Некорректный служебный токен Media type: application/json Example Value | Schema `{ "message": "Entity not found" }`	No links

Вывод

В рамках работы был разработан технический дизайн перехода существующего монолитного backend-приложения к микросервисной архитектуре. Приложение было разделено на четыре независимых сервиса: User Service, Reference Service, Vacancy Service и Response Service. Для каждого сервиса определена собственная база данных в соответствии с принципом database-per-service.

Также были определены способы межсервисного взаимодействия через REST API, спроектированы необходимые внутренние endpoint-ы, описаны форматы запросов и ответов, возможные ошибки и OpenAPI-спецификация.